



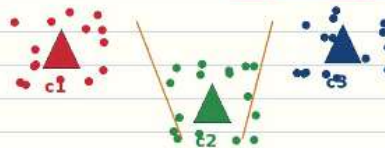
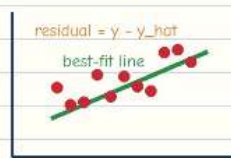
Tajamul Khan

Machine Learning

Handwritten Notes

All Topics Covered

25 beginner-to-advanced ML topics for interview revision



• 25 Topics Covered •

- | | | |
|---------------------------|---------------------------|---------------------------|
| 1. ML Big Picture | 10. KNN + Naive Bayes | 19. Feature Engineering |
| 2. Problem Framing | 11. Decision Trees | 20. Pipelines |
| 3. Linear Regression | 12. Random Forest | 21. Imbalanced Learning |
| 4. Regression Metrics | 13. Gradient Boosting | 22. Model Tuning |
| 5. Regularization | 14. SVM | 23. Interpretability |
| 6. Feature Scaling | 15. PCA | 24. Time Series ML |
| 7. Logistic Regression | 16. Beyond PCA | 25. Interview Cheat Sheet |
| 8. Classification Metrics | 17. K-Means | |
| 9. Threshold Tuning | 18. Hierarchical + DBSCAN | |

by Tajamul Khan

@Tajamulkhann

@Tajamul.codes

1. ML Big Picture

Goal -> choose algorithms by task, data shape and error cost - not by hype.

Learning Types

- Supervised: learn from labelled examples; y is known during training.
- Unsupervised: discover structure without labels - groups, components, density.
- Semi/self-supervised: use weak labels or create a pretext task when labels are costly.

• Interview lens: always state the business objective, target variable, metric and baseline before naming a model.



Model Families

- Linear models = interpretable baseline.
- Trees/ensembles = strong tabular default.
- Kernel methods = powerful on smaller, scaled data.
- Neural nets = flexible when data and compute are large.

Quick revision - fill these gaps in interviews

- Map problem first: target type, prediction time, unit of prediction, and cost of errors.
- Supervised = labels known; unsupervised = structure discovery.
- Strong ML answer = metric + split + baseline + monitoring plan.
- Validation check: compare ML Big Picture with a baseline before claiming improvement.
- Start with a simple baseline before trying complex models.
- If labels are noisy, model quality is limited even with advanced algorithms.
- Interview one-liner: explain where ML Big Picture works, where it fails, and why.
- Error analysis: inspect worst cases and segments, not just the headline metric.

Use when

assumptions match data + metric improves on validation

Avoid when

leakage, bad split or metric does not reflect business cost

2. Problem Framing

Goal -> turn a vague business problem into a measurable ML task.

Define y Carefully

- ◀ Prediction target should match the decision you will automate or support.
- ◀ Set prediction time: what information is available before the decision?
- ◀ Avoid proxy labels that reward the wrong behaviour.

Data Leakage Checklist

- ◀ Future data sneaking into features.
- ◀ Duplicates split across train and test.
- ◀ Target-derived fields such as status_after_approval.
- ◀ Preprocessing fitted on full data instead of train only.

Baseline First

- ◀ Regression: mean/median prediction.
- ◀ Classification: majority class or simple rule.
- ◀ Ranking/recommendation: popularity baseline.

- A simple baseline is not a weak step - it is your proof that the ML model is actually adding value.

Quick revision - fill these gaps in interviews

- ◀ Define the exact prediction moment to prevent future leakage.
- ◀ Check whether false positives or false negatives hurt more.
- ◀ Always build a boring baseline; it exposes whether ML is needed.
- ◀ Validation check: compare Problem Framing with a baseline before claiming improvement.
- ◀ Production check: monitor drift, data quality and metric decay after deployment.
- ◀ Translate business success into a metric before training.
- ◀ Avoid using rows from the same user/entity in both train and test.
- ◀ Interview one-liner: explain where Problem Framing works, where it fails, and why.
- ◀ Error analysis: inspect worst cases and segments, not just the headline metric.
- ◀ Storytelling tip: connect Problem Framing to business cost, not only math.

Use when

assumptions match data + metric improves on validation

Avoid when

leakage, bad split or metric does not reflect business cost



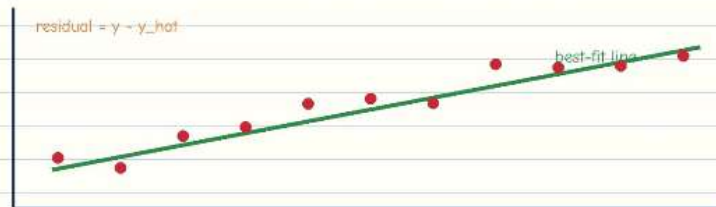
3. Linear Regression

Goal -> predict a continuous value using a weighted combination of features.

Core Idea

$$\hat{y} = b_0 + b_1x_1 + b_2x_2 + \dots + b_px_p$$

- Fit a line/plane that minimises squared residuals.
- Coefficients show expected change in y for a 1-unit change in x , holding other features fixed.
- Works best when signal is close to linear and noise is not extreme.



Training

- Closed-form solution is elegant but can be unstable with many correlated features.
- Gradient descent scales better and is used in many general learners.
- Always inspect residuals; metrics alone can hide patterns.

Quick revision - fill these gaps in interviews

- Use linear regression when relationship is roughly additive and interpretability matters.
- Coefficients explain association, not causation.
- For production, store preprocessing + model together.
- Validation check: compare Linear Regression with a baseline before claiming improvement.
- Production check: monitor drift, data quality and metric decay after deployment.
- Residual plots reveal non-linearity, heteroscedasticity and outliers.
- Normal equation is elegant but gradient descent scales better.
- Interview one-liner: explain where Linear Regression works, where it fails, and why.
- Error analysis: inspect worst cases and segments, not just the headline metric.
- Storytelling tip: connect Linear Regression to business cost, not only math.

Use when

assumptions match data + metric improves on validation

Avoid when

leakage, bad split or metric does not reflect business cost

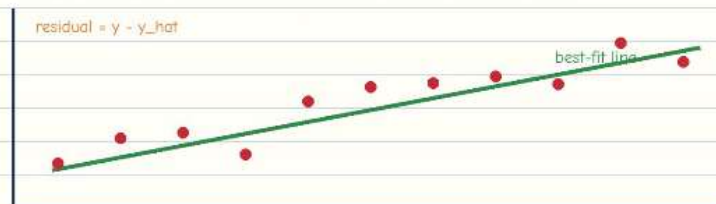


4. Regression Metrics

Goal -> measure prediction error in the same language as the business cost.

Common Metrics

- MAE: average absolute error; robust and easy to explain.
- MSE/RMSE: penalises large errors more strongly.
- R2: variance explained; useful but can look good even with biased predictions.
- MAPE: percentage error; breaks when actual values are near zero.



Residual Diagnostics

- Residuals should look patternless around zero.
- Funnel shape means variance changes with x .
- Curved pattern means the model is missing non-linearity.
- Extreme residuals deserve data-quality and business review.

- For salary, price, revenue and demand forecasting, always translate error into money or operational impact.

Quick revision - fill these gaps in interviews

- RMSE punishes large misses more than MAE.
- R2 can look high while predictions are still useless for decisions.
- Always inspect worst residuals, not just average score.
- Validation check: compare Regression Metrics with a baseline before claiming improvement.
- MAE is easier to explain in business units.
- Use validation error to compare models; use test once at the end.
- Interview one-liner: explain where Regression Metrics works, where it fails, and why.
- Error analysis: inspect worst cases and segments, not just the headline metric.

Use when

assumptions match data + metric improves on validation

Avoid when

leakage, bad split or metric does not reflect business cost



5. Regularization

Goal -> control overfitting by penalising complex coefficient patterns.

◆ Ridge - L2

$$\text{Loss} = \text{MSE} + \lambda \sum w_j^2$$

- ◀ Adds penalty proportional to squared weights.
- ◀ Shrinks correlated features together instead of selecting one.
- ◀ Great default when many weak predictors matter.

◆ Lasso - L1

$$\text{Loss} = \text{MSE} + \lambda \sum |w_j|$$

- ◀ Adds penalty proportional to absolute weights.
- ◀ Can push some coefficients exactly to zero.
- ◀ Useful for sparse feature selection but unstable with highly correlated variables.

◆ Elastic Net

$$\text{Loss} = \text{MSE} + a \cdot \text{L1} + b \cdot \text{L2}$$

- ◀ Mixes L1 and L2; often safer for real-world correlated features.

- Scale numerical features before regularized linear models; otherwise the penalty treats units unfairly.

◆ Quick revision - fill these gaps in interviews

- ◀ Ridge keeps all features but shrinks unstable weights.
- ◀ Elastic Net is safer when features are correlated.
- ◀ Scale features before applying any coefficient penalty.
- ◀ Validation check: compare Regularization with a baseline before claiming improvement.
- ◀ Lasso can remove features by setting weights to zero.
- ◀ Lambda too high causes underfitting; too low behaves like plain regression.
- ◀ Interview one-liner: explain where Regularization works, where it fails, and why.
- ◀ Error analysis: inspect worst cases and segments, not just the headline metric.

Use when

assumptions match data + metric improves on validation

Avoid when

leakage, bad split or metric does not reflect business cost

6. Feature Scaling

Goal -> put features on comparable scales when models depend on distance, margin or penalties.

◆ When Scaling Matters

- ◀ Linear/logistic regression with regularization.
- ◀ KNN, K-Means, SVM, PCA and neural networks.
- ◀ Gradient descent becomes smoother and faster.



◆ Common Scalers

- ◀ StandardScaler: mean 0, std 1 - strong default.
- ◀ MinMaxScaler: fixed range - useful for bounded inputs.
- ◀ RobustScaler: median/IQR - better with outliers.

◆ Do It Safely

- ◀ Fit scaler only on training data.
- ◀ Apply the same fitted scaler to validation/test/production.
- ◀ Use a Pipeline to prevent leakage.

◆ Quick revision - fill these gaps in interviews

- | | |
|--|--|
| ◀ Distance-based models become unfair when units differ. | ◀ Fit scaler only on training data; transform validation/test with same scaler. |
| ◀ RobustScaler is useful when outliers are present. | ◀ Tree models usually do not need scaling. |
| ◀ Put scaling inside a pipeline to make deployment reproducible. | ◀ Interview one-liner: explain where Feature Scaling works, where it fails, and why. |
| ◀ Validation check: compare Feature Scaling | ◀ Error analysis: inspect worst cases and segments, not just the headline metric. |

Use when

assumptions match data + metric improves on validation

Avoid when

leakage, bad split or metric does not reflect business cost

7. Logistic Regression

Goal -> estimate class probability using a linear score passed through sigmoid.

Binary Classifier

$$p = 1 / (1 + \exp(-(w^T x + b)))$$

- Outputs $P(y=1|x)$, not just a class label.
- Decision boundary is linear in feature space.
- Coefficients can be interpreted through log-odds.



Training Loss

$$\text{LogLoss} = -\text{mean}[y \log(p) + (1-y) \log(1-p)]$$

- Cross-entropy punishes confident wrong predictions heavily.
- Threshold 0.5 is only a default, not a law.
- Calibrate probabilities when decisions depend on risk scores.

Quick revision - fill these gaps in interviews

- Logistic regression is a probability model, not only a classifier.
- Coefficients work in log-odds; exponentiating gives odds ratios.
- Regularization helps when many correlated predictors exist.
- Validation check: compare Logistic Regression with a baseline before claiming improvement.
- The boundary is linear unless features are transformed.
- Tune the threshold based on business risk.
- Interview one-liner: explain where Logistic Regression works, where it fails, and why.
- Error analysis: inspect worst cases and segments, not just the headline metric.
- Storytelling tip: connect Logistic Regression to

Use when

assumptions match data + metric improves on validation

Avoid when

leakage, bad split or metric does not reflect business cost



8. Classification Metrics

Goal -> choose metrics based on which mistakes are more expensive.

Confusion Matrix

- TP: model correctly catches positive cases.
- FP: false alarm.
- FN: missed positive case.
- TN: correctly ignored negative case.

Metric Choice

- Precision: among predicted positives, how many were correct?
- Recall: among real positives, how many did we catch?
- F1: balances precision and recall.
- PR-AUC is better than ROC-AUC when positives are rare.

	pred +	pred -	
actual +	TP	FN	accuracy = (TP + TN) / all
actual -	FP	TN	precision = TP / (TP + FP)
			recall = TP / (TP + FN)
			F1 = 2PR / (P + R)

- For fraud, disease, churn and safety cases, accuracy can be misleading. Start with recall, precision and business cost.

Quick revision - fill these gaps in interviews

- Accuracy hides failure on imbalanced datasets.
- Recall answers: how many true positives did we catch?
- PR-AUC is usually more informative when positives are rare.
- Validation check: compare Classification Metrics with a baseline before claiming
- Precision answers: how trustworthy are positive predictions?
- F1 is useful when precision and recall both matter.
- Interview one-liner: explain where Classification Metrics works, where it fails, and
- Error analysis: inspect worst cases and

Use when

assumptions match data + metric improves on validation

Avoid when

leakage, bad split or metric does not reflect business cost

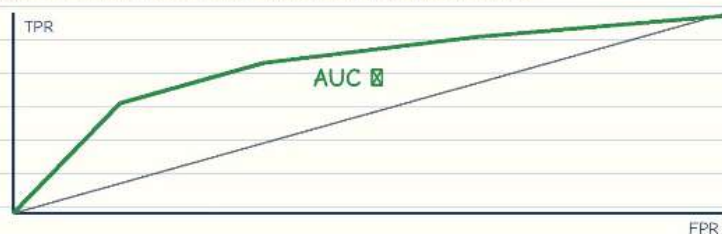


9. Threshold Tuning

Goal -> convert probabilities into actions with a threshold matched to business cost.

Why Tune Thresholds?

- ◀ The model learns probabilities; the business needs decisions.
- ◀ Lower threshold increases recall but often lowers precision.
- ◀ Higher threshold reduces false alarms but may miss positives.



How to Pick It

- ◀ Use validation data, not test data.
- ◀ Plot precision-recall across thresholds.
- ◀ Maximise expected value or minimise cost when cost matrix is known.
- ◀ Freeze threshold before final test evaluation.

Quick revision - fill these gaps in interviews

- ◀ Threshold 0.5 is a default, not a business rule.
- ◀ Higher threshold reduces false positives but misses more cases.
- ◀ Use expected cost/profit when error costs are known.
- ◀ Validation check: compare Threshold Tuning with a baseline before claiming improvement.
- ◀ Production check: monitor drift, data quality and metric decay after deployment.
- ◀ Lower threshold catches more positives but creates more alarms.
- ◀ Pick threshold on validation data, then freeze it.
- ◀ Interview one-liner: explain where Threshold Tuning works, where it fails, and why.
- ◀ Error analysis: inspect worst cases and segments, not just the headline metric.
- ◀ Storytelling tip: connect Threshold Tuning to business cost, not only math.

Use when

assumptions match data + metric improves on validation

Avoid when

leakage, bad split or metric does not reflect business cost



10. KNN and Naive Bayes

Goal -> two simple models that are still useful in the right setting.

◆ K-Nearest Neighbours

- ◀ Predict from the majority/average of nearby examples.
- ◀ No real training phase - prediction can be slow.
- ◀ Sensitive to scaling, irrelevant features and high dimensions.
- ◀ Good for small clean datasets and local patterns.

◆ Naive Bayes

$$P(\text{class}|x) \propto P(\text{class}) * \text{product } P(x_j|\text{class})$$

- ◀ Uses Bayes rule with conditional-independence assumption.
- ◀ Very fast and strong for text classification baselines.
- ◀ Laplace smoothing avoids zero probability issues.
- ◀ Often gives surprisingly good results with TF-IDF/count features.

- Simple models are excellent interview answers when you explain assumptions and failure cases clearly.

◆ Quick revision - fill these gaps in interviews

- ◀ KNN needs scaling and becomes weak in high dimensions.
- ◀ Naive Bayes is fast and strong for text baselines.
- ◀ Use smoothing to avoid zero probabilities.
- ◀ Validation check: compare KNN and Naive Bayes with a baseline before claiming
- ◀ Production check: monitor drift, data quality and metric decay after deployment.
- ◀ Choose k with validation; tiny k overfits, huge k oversmooths.
- ◀ Independence assumption is naive, but often practical.
- ◀ Interview one-liner: explain where KNN and Naive Bayes works, where it fails, and why.
- ◀ Error analysis: inspect worst cases and segments, not just the headline metric.
- ◀ Storytelling tip: connect KNN and Naive Bayes to business cost, not only math.

Use when

assumptions match data + metric improves on validation

Avoid when

leakage, bad split or metric does not reflect business cost

11. Decision Trees

Goal -> split feature space using simple rules until regions become pure enough.

Split Criteria

Gini = $1 - \sum(p_k^2)$ | Entropy = $-\sum(p_k \log_2 p_k)$

- ◀ Gini and entropy both measure class impurity.
- ◀ Regression trees usually split by variance/MSE reduction.
- ◀ Trees are greedy: each split is locally best, not globally best.



Overfitting Controls

- ◀ max_depth, min_samples_leaf, min_samples_split.
- ◀ Pruning trims branches that do not help validation performance.
- ◀ Single trees are interpretable but high variance.

Quick revision - fill these gaps in interviews

- ◀ Decision trees are easy to explain but easy to overfit.
- ◀ Greedy splits do not guarantee a globally best tree.
- ◀ Prune or limit depth to improve generalization.
- ◀ No scaling needed for tree splits.
- ◀ For regression trees, leaves predict average target value.
- ◀ Interview one-liner: explain where Decision Trees works, where it fails, and why.
- ◀ Validation check: compare Decision Trees with a baseline before claiming improvement.
- ◀ Error analysis: inspect worst cases and segments, not just the headline metric.
- ◀ Production check: monitor drift, data quality and metric decay after deployment.
- ◀ Storytelling tip: connect Decision Trees to business cost, not only math.

Use when

assumptions match data + metric improves on validation

Avoid when

leakage, bad split or metric does not reflect business cost

12. Random Forest

Goal -> combine many de-correlated trees to reduce variance.

How It Works

- ▶ Bootstrap samples create different training sets.
- ▶ Random feature subsets create diverse trees.
- ▶ Average/vote across trees gives stable predictions.



Strengths

- ▶ Strong tabular baseline with little preprocessing.
- ▶ Handles non-linearity and interactions.
- ▶ Less sensitive to outliers than linear models.

Limitations

- ▶ Less interpretable than one tree.
- ▶ Can be bulky and slower to serve.
- ▶ Feature importance can be biased toward high-cardinality features.

Quick revision - fill these gaps in interviews

- ▶ Random Forest reduces variance by averaging many diverse trees.
- ▶ Feature importance can mislead with correlated/high-cardinality features.
- ▶ Use out-of-bag score as an internal validation signal when appropriate.
- ▶ Validation check: compare Random Forest
- ▶ Great first model for messy tabular data.
- ▶ More trees improve stability but increase serving cost.
- ▶ Interview one-liner: explain where Random Forest works, where it fails, and why.
- ▶ Error analysis: inspect worst cases and segments, not just the headline metric.

Use when

assumptions match data + metric improves on validation

Avoid when

leakage, bad split or metric does not reflect business cost



13. Gradient Boosting

Goal -> build trees sequentially so each new tree corrects previous errors.

Core Intuition

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

- ◀ Bagging reduces variance; boosting reduces bias step by step.
- ◀ Each tree fits residuals/gradients from the current ensemble.
- ◀ Learning rate controls how strongly each tree contributes.

Popular Libraries

- ◀ XGBoost: regularized boosting, strong general-purpose choice.
- ◀ LightGBM: fast histogram-based training, great on large tabular data.
- ◀ CatBoost: strong handling of categorical features.

Tuning Levers

- ◀ `n_estimators`, `learning_rate`, `max_depth/num_leaves`, `subsample`, `colsample`, `regularization`.

- A smaller learning rate usually needs more trees but can generalise better.

Quick revision - fill these gaps in interviews

- ◀ Boosting learns sequentially; each tree fixes previous mistakes.
- ◀ Depth/num_leaves controls interaction complexity.
- ◀ For tabular ML, gradient boosting is often the strongest baseline.
- ◀ Validation check: compare Gradient Boosting with a baseline before claiming improvement.
- ◀ Production check: monitor drift, data quality and metric decay after deployment.
- ◀ Small learning rate often generalizes better but needs more trees.
- ◀ Early stopping prevents unnecessary overfitting.
- ◀ Interview one-liner: explain where Gradient Boosting works, where it fails, and why.
- ◀ Error analysis: inspect worst cases and segments, not just the headline metric.
- ◀ Storytelling tip: connect Gradient Boosting to business cost, not only math.

Use when

assumptions match data + metric improves on validation

Avoid when

leakage, bad split or metric does not reflect business cost



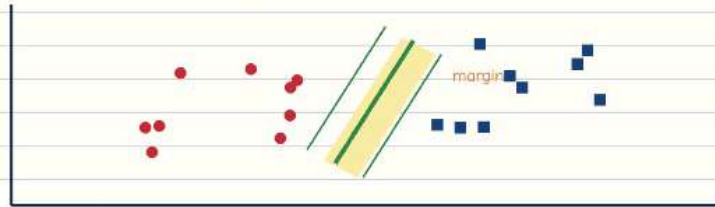
14. Support Vector Machines

Goal -> find a separating boundary with the largest possible margin.

Margin Thinking

$$\text{Hard margin: } y_i(w^T x_i + b) \geq 1$$

- Only support vectors close to the boundary define the solution.
- Large margin usually improves generalisation.
- C controls tolerance for margin violations.



Kernel Trick

$$K_{\text{RBF}}(x, z) = \exp(-\gamma \|x - z\|^2)$$

- Linear kernel for simple separable data.
- RBF kernel creates non-linear boundaries without explicit feature mapping.
- Gamma controls how local the RBF influence is.

- SVMs need scaling. They shine on small-to-medium datasets; they can become slow on very large data.

Quick revision - fill these gaps in interviews

- SVMs maximize margin, only support vectors drive the boundary.
- Gamma high = very local RBF boundary, higher overfit risk.
- Best for small-to-medium datasets, not massive row counts.
- Validation check: compare Support Vector Machines with a baseline before claiming.
- C high = fewer violations, higher overfit risk.
- Always scale features before SVM.
- Interview one-liner: explain where Support Vector Machines works, where it fails, and why.
- Error analysis: inspect worst cases and segments, not just the headline metric.



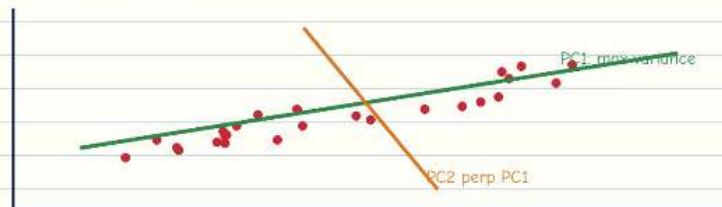
15. PCA

Goal -> reduce dimensions by projecting data onto directions of maximum variance.

Recipe

$$Z = X_{\text{centered}} * V_k$$

- Center data by subtracting feature means.
- Scale features when units differ.
- Compute directions of maximum variance using eigenvectors or SVD.
- Project data onto top k components.



Explained Variance

- Explained variance ratio tells how much information each component carries.
- Use scree plot and cumulative variance to choose k.
- Components are combinations of original features, not original features.

Quick revision - fill these gaps in interviews

- PCA finds directions of maximum variance, not most predictive features.
- Use cumulative explained variance to choose k.
- Do not interpret PCs as causal drivers.
- Validation check: compare PCA with a baseline before claiming improvement.
- Production check: monitor drift, data quality and metric decay after deployment.
- Always center data; scale when feature units differ.
- PCA components are linear combinations of original features.
- Interview one-liner: explain where PCA works, where it fails, and why.
- Error analysis: inspect worst cases and segments, not just the headline metric.
- Storytelling tip: connect PCA to business cost, not only math.

Use when

assumptions match data + metric improves on validation

Avoid when

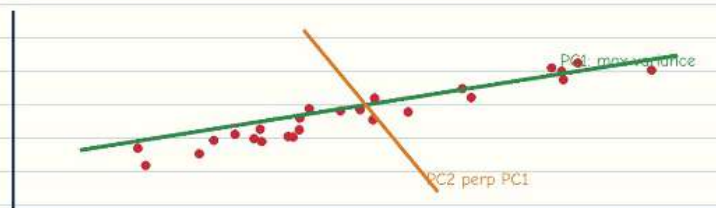
leakage, bad split or metric does not reflect business cost

16. Beyond PCA

Goal -> use non-linear embeddings carefully for visualisation and neighbourhood discovery.

◆ t-SNE

- Excellent for 2D visualisation of local clusters.
- Distances between far clusters are not reliable.
- Not ideal as a general preprocessing step for supervised ML.



◆ UMAP

- Often preserves more global structure than t-SNE.
- Can transform new points if fitted appropriately.
- Hyperparameters `n_neighbors` and `min_dist` change the story.

◆ Autoencoders

- Neural networks learn compressed representations.
- Useful when data is large, noisy or non-linear.
- Needs validation – pretty embeddings can still be misleading.

- Use PCA for reliable linear compression; use t-SNE/UMAP mainly to explore and communicate structure.

◆ Quick revision – fill these gaps in interviews

- t-SNE is mainly for visualization, not reliable global distances.
- UMAP often preserves more neighbourhood structure.
- Autoencoders can learn non-linear compression but need enough data.
- Embeddings can look clean even when labels are messy.
- Always validate downstream impact, not just visual appeal.
- Interview one-liner: explain where Beyond PCA works, where it fails, and why.

17. K-Means Clustering

Goal -> partition data into k compact groups by minimising within-cluster distance.

◆ Lloyd Algorithm

minimize $\sum_k \sum_{x \in C_k} ||x - \mu_k||^2$

- ▶ Pick k initial centroids.
- ▶ Assign each point to nearest centroid.
- ▶ Move each centroid to the mean of assigned points.
- ▶ Repeat until assignments stabilize.



◆ Choosing k

- ▶ Elbow plot: look for diminishing returns.
- ▶ Silhouette score: separation and cohesion.
- ▶ Business interpretability: clusters must be useful, not just mathematical.

- Use k-means++ initialization and scale features before clustering.

◆ Quick revision - fill these gaps in interviews

- ▶ K-Means prefers spherical, similar-sized clusters.
- ▶ k-means++ improves centroid initialization.
- ▶ Cluster names should be based on feature profiles, not algorithm IDs.
- ▶ Validation check: compare K-Means Clustering with a baseline before claiming improvement.
- ▶ Scale features before measuring distance.
- ▶ Elbow method is subjective; use silhouette and domain meaning too.
- ▶ Interview one-liner: explain where K-Means Clustering works, where it fails, and why.
- ▶ Error analysis: inspect worst cases and segments, not just the headline metric.

Use when

assumptions match data + metric improves on validation

Avoid when

leakage, bad split or metric does not reflect business cost



18. Hierarchical and DBSCAN

Goal -> cluster either by building a tree or by finding dense regions.

◆ Hierarchical Clustering

- ◀ Agglomerative: start with each point, merge nearest clusters.
- ◀ Linkage options: single, complete, average, ward.
- ◀ Dendrogram helps choose a cut level.

◆ DBSCAN

- ◀ Groups dense regions separated by sparse areas.
- ◀ eps controls radius; min_samples controls density threshold.
- ◀ Finds arbitrary shapes and marks noise points.
- ◀ Struggles when densities vary a lot.

- Rule of thumb: spherical known-k groups -> K-Means; irregular shapes/noise -> DBSCAN; hierarchy needed -> agglomerative.

◆ Quick revision - fill these gaps in interviews

- ◀ Hierarchical clustering gives a dendrogram for multiple cut levels.
- ◀ DBSCAN finds irregular shapes and noise.
- ◀ When density varies a lot, DBSCAN can fail badly.
- ◀ Validation check: compare Hierarchical and DBSCAN with a baseline before claiming.
- ◀ Production check: monitor drift, data quality and metric decay after deployment.
- ◀ Ward linkage often works well for compact clusters.
- ◀ eps is the hardest DBSCAN parameter.
- ◀ Interview one-liner: explain where Hierarchical and DBSCAN works, where it fails, and why.
- ◀ Error analysis: inspect worst cases and segments, not just the headline metric.
- ◀ Storytelling tip: connect Hierarchical and DBSCAN to business cost, not only math.

Use when

assumptions match data + metric improves on validation

Avoid when

leakage, bad split or metric does not reflect business cost

19. Feature Engineering

Goal -> make raw data easier for models to learn from.

Numerical Features

- ◀ Ratios, differences, logs, bins and interactions can reveal signal.
- ◀ Winsorize or cap extreme values only when business logic supports it.
- ◀ Missingness indicators can be predictive.

Categorical Features

- ◀ One-hot for low cardinality.
- ◀ Ordinal encoding only when order is meaningful.
- ◀ Target encoding must be done inside cross-validation to avoid leakage.

Datetime Features

- ◀ Extract day/week/month, seasonality, holidays and recency.
- ◀ Use rolling aggregates only from past data.

- Great feature engineering often beats fancy algorithms on tabular business data.

Quick revision - fill these gaps in interviews

- ◀ Feature engineering should follow business logic, not random creativity.
- ◀ Target encoding must be done inside cross-validation.
- ◀ A simple useful feature can beat a complex algorithm.
- ◀ Validation check: compare Feature Engineering with a baseline before claiming.
- ◀ Production check: monitor drift, data quality and metric decay after deployment.
- ◀ Datetime leakage is common with rolling averages.
- ◀ Missingness itself can carry signal.
- ◀ Interview one-liner: explain where Feature Engineering works, where it fails, and why.
- ◀ Error analysis: inspect worst cases and segments, not just the headline metric.
- ◀ Storytelling tip: connect Feature Engineering to business cost, not only math.

Use when

assumptions match data + metric improves on validation

Avoid when

leakage, bad split or metric does not reflect business cost



20. Preprocessing Pipelines

Goal -> keep transformations reproducible, leak-free and production-ready.

Missing Values

- Mean/median for numerical baselines; mode or explicit Unknown for categorical.
- Model-based imputation can help but must be validated.
- Never impute using test-set information.



Pipelines

- Chain imputation, scaling, encoding and model training.
- Fit pipeline on train; transform validation/test.
- Save the whole pipeline object, not just the model.

Train/Validation/Test

- Train learns parameters.
- Validation selects model and threshold.
- Test estimates final generalisation once.

Quick revision - fill these gaps in interviews

- Pipelines prevent train-test preprocessing mismatch.
- Fit transformations only on train data.
- Make preprocessing deterministic before deployment.
- Validation check: compare Preprocessing Pipelines with a baseline before claiming.
- Save the full pipeline, not only model weights.
- Use ColumnTransformer for numeric + categorical flows.
- Interview one-liner: explain where Preprocessing Pipelines works, where it fails.
- Error analysis: inspect worst cases and segments, not just the headline metric.

Use when

assumptions match data + metric improves on validation

Avoid when

leakage, bad split or metric does not reflect business cost

21. Imbalanced Learning

Goal -> handle rare positive classes without fooling yourself with accuracy.

What Changes?

- Accuracy becomes misleading.
- The minority class may have very few examples for learning patterns.
- Probability calibration can shift after resampling.

Useful Fixes

- Class weights or cost-sensitive learning.
- Undersampling majority / oversampling minority.
- SMOTE for synthetic minority examples - validate carefully.
- Tune threshold using PR curve and business cost.

Evaluation

- Use stratified splits.
- Prefer PR-AUC, recall@precision, precision@k or cost-based metrics.
- Check confusion matrix at the chosen threshold.

	pred +	pred -	
actual +	TP	FN	$accuracy = (TP + TN) / all$
actual -	FP	TN	$precision = TP / (TP + FP)$
			$recall = TP / (TP + FN)$
			$F1 = 2PR / (P + R)$

Quick revision - fill these gaps in interviews

- Rare positive classes need metrics beyond accuracy.
- SMOTE should be applied only inside training folds.
- Evaluate at the operating point the business will actually use.
- Validation check: compare Imbalanced
- Class weights are often the cleanest first fix.
- Threshold tuning is often more powerful than resampling.
- Interview one-liner: explain where Imbalanced Learning works, where it fails, and why.
- Error analysis: inspect worst cases and segments, not just the headline metric.

Use when

assumptions match data + metric improves on validation

Avoid when

leakage, bad split or metric does not reflect business cost

22. Model Selection and Tuning

Goal -> search for good hyperparameters without leaking test information.

Validation Strategy

- Random split for iid data.
- Stratified split for class imbalance.
- Group split when same customer/user/entity appears multiple times.
- Time-based split for time series.

Search Methods

- Grid search: systematic but expensive.
- Random search: strong baseline for wide spaces.
- Bayesian/Optuna: efficient guided search.
- Early stopping: stop when validation stops improving.

Nested CV

- Use when you need an unbiased estimate after tuning, especially for small datasets.

- Never tune hyperparameters on the final test set. The test set is for the final audit, not experimentation.

Quick revision - fill these gaps in interviews

- Choose split strategy based on data generation process.
- Random search often beats grid search for wide spaces.
- Never repeatedly check the final test score while tuning.
- Validation check: compare Model Selection and Tuning with a baseline before claiming.
- Production check: monitor drift, data quality and metric decay after deployment.
- Group leakage happens when same entity appears in multiple splits.
- Use early stopping to save time and reduce overfit.
- Interview one-liner: explain where Model Selection and Tuning works, where it fails, and
- Error analysis: inspect worst cases and segments, not just the headline metric.
- Storytelling tip: connect Model Selection and Tuning to business cost, not only math.

Use when

assumptions match data + metric improves on validation

Avoid when

leakage, bad split or metric does not reflect business cost



23. Interpretability

Goal -> explain model behaviour globally and locally without overclaiming.

Global Explanations

- ◀ Permutation importance: drop in performance when a feature is shuffled.
- ◀ Partial Dependence: average effect of a feature.
- ◀ Feature importance in trees is quick but can be biased.

Local Explanations

- ◀ SHAP explains one prediction as feature contributions.
- ◀ LIME approximates local behaviour around a point.
- ◀ Counterfactuals answer: what minimal change flips the decision?

Caution

- ◀ Correlation is not causation.
- ◀ Explanations can be unstable with correlated features.
- ◀ Always pair model explanations with domain review.

- In interviews, explain not only what feature matters, but whether that feature is actionable and fair.

Quick revision - fill these gaps in interviews

- ◀ Interpretability is not the same as causality.
- ◀ SHAP can explain one prediction but may be unstable with correlated features.
- ◀ Pair explanations with error analysis and domain review.
- ◀ Validation check: compare Interpretability with a baseline before claiming improvement.
- ◀ Production check: monitor drift, data quality and metric decay after deployment.
- ◀ Permutation importance measures performance drop after shuffling.
- ◀ Check whether important features are actionable.
- ◀ Interview one-liner: explain where Interpretability works, where it fails, and why.
- ◀ Error analysis: inspect worst cases and segments, not just the headline metric.
- ◀ Storytelling tip: connect Interpretability to business cost, not only math.

Use when

assumptions match data + metric improves on validation

Avoid when

leakage, bad split or metric does not reflect business cost

24. Time Series ML

Goal -> predict the future using only information that existed in the past.

Feature Ideas

- ◀ Lag features: $y(t-1)$, $y(t-7)$, $y(t-30)$.
- ◀ Rolling statistics: mean, max, std over previous windows.
- ◀ Calendar effects: weekday, month, holiday, promotion, season.



Validation

- ◀ Use walk-forward validation.
- ◀ Do not shuffle time series randomly.
- ◀ Backtest across multiple periods to check stability.

Leakage Traps

- ◀ Rolling window accidentally includes current/future value.
- ◀ Using revised data not available at prediction time.
- ◀ Aggregates computed over the full dataset.

Quick revision - fill these gaps in interviews

- ◀ Never shuffle time series for final validation.
- ◀ Rolling features must exclude the current/future target.
- ◀ A naive seasonal baseline is mandatory before complex ML.
- ◀ Validation check: compare Time Series ML with a baseline before claiming improvement.
- ◀ Create lags only from past values.
- ◀ Backtest across multiple windows to test stability.
- ◀ Interview one-liner: explain where Time Series ML works, where it fails, and why.
- ◀ Error analysis: inspect worst cases and segments, not just the headline metric.

Use when

assumptions match data + metric improves on validation

Avoid when

leakage, bad split or metric does not reflect business cost



25. Interview Cheat Sheet

Goal -> select a strong starting model and explain why.

Algorithm Selection

- Linear/logistic: interpretable baseline.
- Random Forest: robust tabular default.
- Gradient boosting: high-performance tabular workhorse.
- SVM/KNN: smaller scaled datasets.
- PCA/K-Means/DBSCAN: unsupervised structure.

Final ML Checklist

- Define target and prediction time.
- Create leak-free split.
- Build baseline.
- Use pipeline.
- Pick metric tied to business cost.
- Tune on validation only.
- Inspect errors and monitor after deployment.

Task	Start Here	Try Next	Watch Out
Regression	Linear/Ridge	RF/GBM/XGBoost	leakage + outliers
Classification	Logistic	RF/SVM/Boosting	imbalance
Clustering	K-Means	DBSCAN/GMM	unknown k
High-Dim	PCA	UMAP/t-SNE	scaling
Time Series	Naive baseline	GBM/lags	future leakage

- Best answer format: problem -> metric -> split -> baseline -> model -> validation -> error analysis -> deployment monitoring.

Quick revision - fill these gaps in interviews

- Interview flow: problem -> metric -> split -> baseline -> model -> validation -> deployment.
- Explain why a model fits the data and business constraint.
- Mention leakage, imbalance and monitoring proactively.
- Start simple; justify complexity only when it improves validation.